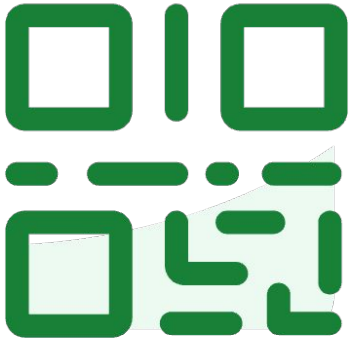
Lecture 2

Data Tools

Tools we need for ML – `pandas` and visualization

EECS 189/289, Fall 2025 @ UC Berkeley
Joseph E. Gonzalez and Narges Norouzi



Join at slido.com
#7824888

Why Start with Data?

7824888

- **Data is the foundation of ML**

Every model begins and ends with data. It's the raw material for training and evaluation.

- **Inputs and Experiments**

Success in ML depends on how well we process inputs *and* outputs from experiments.

- **Critical Skill for Research & Industry**

Whether you join a lab or work in industry, strong data wrangling and visualization skills are essential.

Roadmap

7824888

- pandas
  - pandas Data Structures
  - Exploring DataFrames
  - Selecting and Retrieving Data in DataFrames
  - Filtering Data
  - DataFrame Modification
  - Aggregation in DataFrame
  - Joining DataFrames
- Visualization
  - Matplotlib
  - Plotly

pandas Data Structures

7824888

pandas

Stands for "panel data"

7824888

Using pandas, we can:

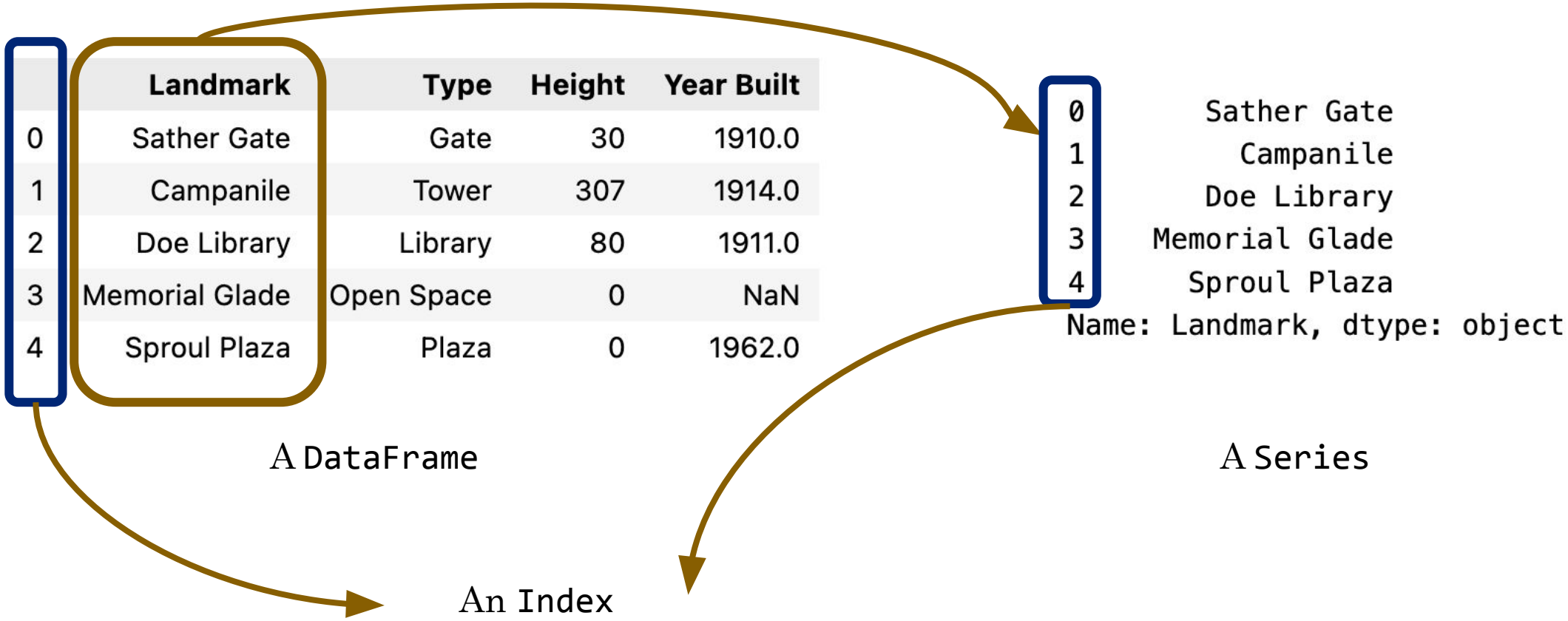
- Arrange data in a tabular format.
- Extract useful information filtered by specific conditions.
- Operate on data to gain new information.
- Apply numerical operations using `NumPy` to our data.
- Perform vectorized computations to speed up our analysis.

`pandas` is the standard tool across research and industry for working with tabular data.

pandas Data Types

7824888

- In the language of `pandas`, tables are referred to as `DataFrames`.



	Landmark	Type	Height	Year Built
0	Sather Gate	Gate	30	1910.0
1	Campanile	Tower	307	1914.0
2	Doe Library	Library	80	1911.0
3	Memorial Glade	Open Space	0	NaN
4	Sproul Plaza	Plaza	0	1962.0

A `DataFrame`

```
0       Sather Gate
1         Campanile
2       Doe Library
3    Memorial Glade
4       Sproul Plaza
Name: Landmark, dtype: object
```

A `Series`

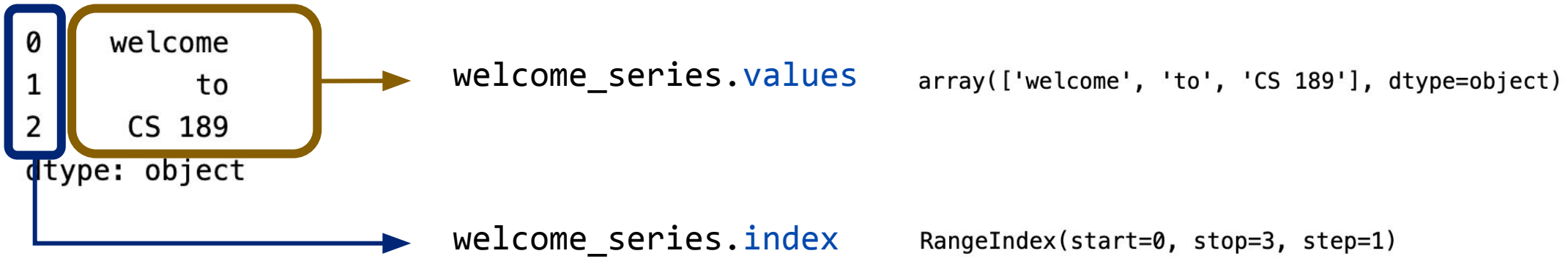
An `Index`

Series

7824888

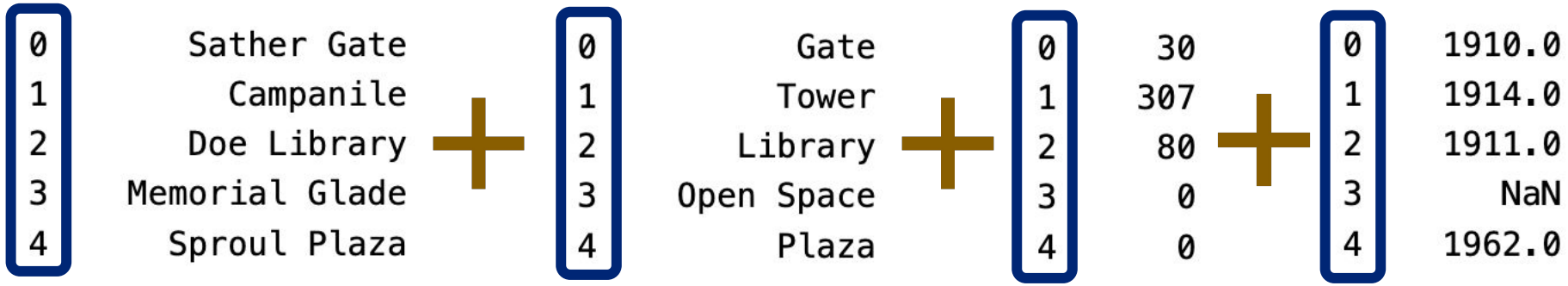
- A `Series` is a one-dimensional labeled array.
- Components of a `Series` object:
  - **`Values`**: The actual data stored in the `Series`.
  - **`Index`**: The labels associated with each data point, which allow for easy access and manipulation.

```
import pandas as pd
welcome_series = pd.Series(["welcome", "to", "CS 189"])
```



DataFrame

7824888



	Landmark	Type	Height	Year Built
0	Sather Gate	Gate	30	1910.0
1	Campanile	Tower	307	1914.0
2	Doe Library	Library	80	1911.0
3	Memorial Glade	Open Space	0	NaN
4	Sproul Plaza	Plaza	0	1962.0

You can create a `DataFrame`:

- From a CSV file
- Using a dictionary
- Using a list and column names
- From `Series`

Creating a DataFrame

7824888

```
data = {
'Landmark': ['Sather Gate', 'Campanile', 'Doe Library', 'Memorial Glade', 'Sproul Plaza'],
'Type': ['Gate', 'Tower', 'Library', 'Open Space', 'Plaza'],
'Height': [30, 307, 80, 0, 0],
'Year Built': [1910, 1914, 1911, None, 1962]
}
df = pd.DataFrame(data)
```

	Landmark	Type	Height	Year Built
0	Sather Gate	Gate	30	1910.0
1	Campanile	Tower	307	1914.0
2	Doe Library	Library	80	1911.0
3	Memorial Glade	Open Space	0	NaN
4	Sproul Plaza	Plaza	0	1962.0

```
event_data = pd.read_csv("data/uc_berkeley_events.csv", index_col='Year')
```

Year	Event	Location
1868	Founding of UC Berkeley	Berkeley, CA
1914	Completion of Campanile	Berkeley, CA
1923	Opening of Memorial Stadium	Berkeley, CA
1964	Free Speech Movement	Berkeley, CA
2000	Opening of Hearst Memorial Mining Building	Berkeley, CA

Exploring DataFrames

7824888

- pandas
  - pandas Data Structures
  - Exploring DataFrames
  - Selecting and Retrieving Data in DataFrames
  - Filtering Data
  - DataFrame Modification
  - Aggregation in DataFrame
  - Joining DataFrames
- Visualization
  - Matplotlib
  - Plotly

Utility Functions for `DataFrame`

7824888

- Understanding the structure and content of `DataFrame` is an essential first step in data analysis. Here are some methods to get a quick overview of `DataFrame`:
  - `head()`and `tail()`,
  - `info()`,
  - `describe()`,
  - `sample()`,
  - `value_counts()`, and
  - `unique()`.

`head()` and `tail()`

7824888

	Landmark	Type	Height	Year Built
0	Sather Gate	Gate	30	1910.0
1	Campanile	Tower	307	1914.0
2	Doe Library	Library	80	1911.0
3	Memorial Glade	Open Space	0	NaN
4	Sproul Plaza	Plaza	0	1962.0

Use **`head()`** to display the first few rows of the `DataFrame`.

```
df.head() # default is 5
```

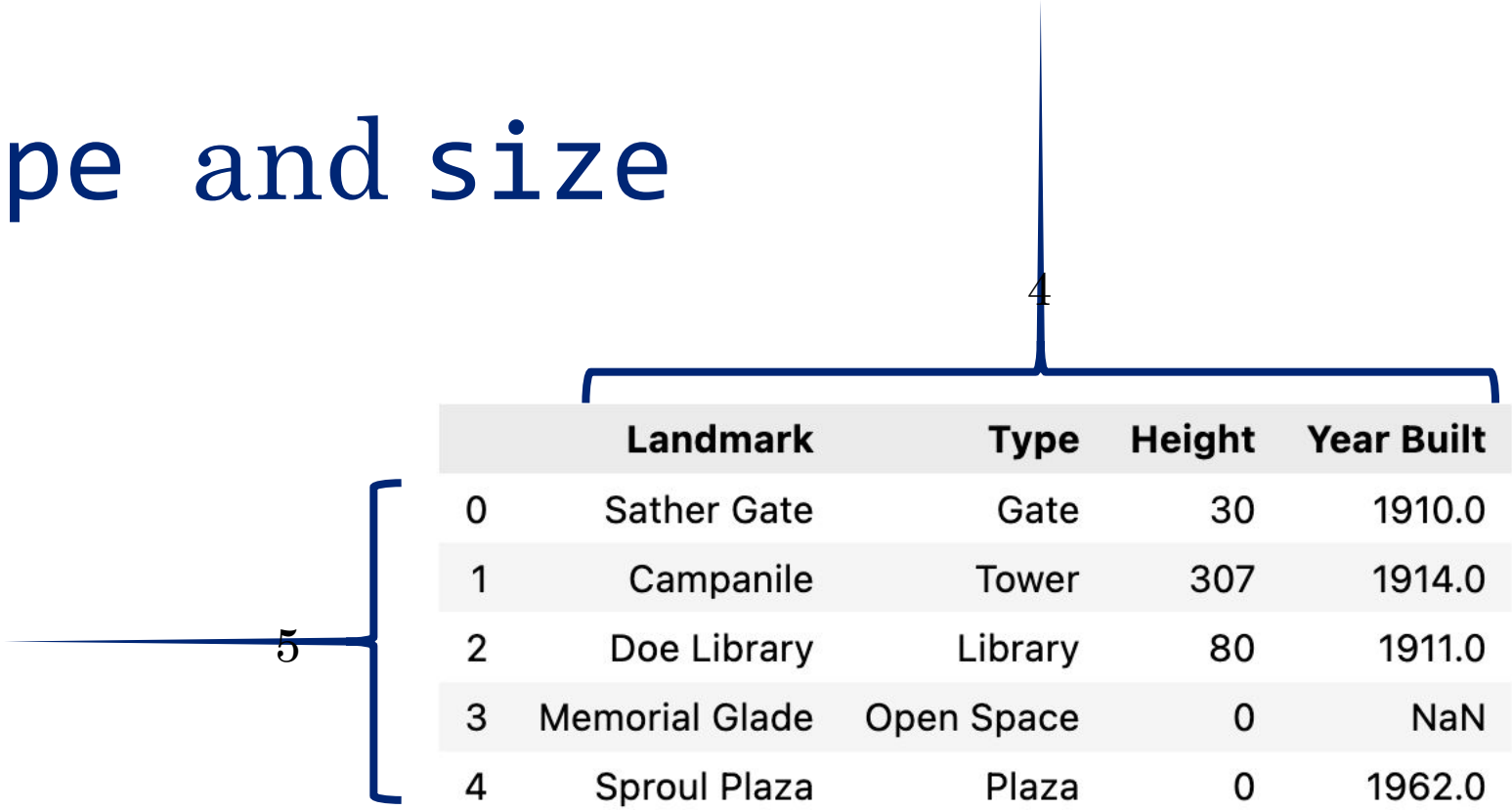
	Landmark	Type	Height	Year Built
0	Sather Gate	Gate	30	1910.0
1	Campanile	Tower	307	1914.0
2	Doe Library	Library	80	1911.0
3	Memorial Glade	Open Space	0	NaN
4	Sproul Plaza	Plaza	0	1962.0

Use **`tail()`** to display the last few rows

```
df.tail(3)
```

	Landmark	Type	Height	Year Built
2	Doe Library	Library	80	1911.0
3	Memorial Glade	Open Space	0	NaN
4	Sproul Plaza	Plaza	0	1962.0

shape and size



	Landmark	Type	Height	Year Built
0	Sather Gate	Gate	30	1910.0
1	Campanile	Tower	307	1914.0
2	Doe Library	Library	80	1911.0
3	Memorial Glade	Open Space	0	NaN
4	Sproul Plaza	Plaza	0	1962.0

`shape` gets the number of rows and columns in the `DataFrame`.

```
df.shape
```

```
(5, 4)
```

$5 \times 4 = 20$

`size` gets the total number of elements in the `DataFrame`.

```
df.size
```

```
20
```

7824888

`sample()`

To sample a random selection of rows from a `DataFrame`, we use the `.sample()` method.

`.sample(n=#)`

Randomly sample `n` rows.

`.sample(frac=#)`

Randomly sample `frac` of rows.

`.sample(n=#, replace=True)`

Allows the same row to appear multiple times.

`.sample(n=#, random_state=42)`

Using `random_state` for reproducibility.

value_counts() and unique()

7824888

Series.value_counts

Counts the number of occurrences of each unique value in a Series.

```
type_counts = df['Type'].value_counts()
```

```
Type
Gate          1
Tower         1
Library       1
Open Space    1
Plaza         1
Name: count, dtype: int64
```

Series.unique

Returns an array of every unique value in a Series.

```
type_counts = df['Type'].unique()
```

```
['Gate' 'Tower' 'Library' 'Open Space' 'Plaza']
```

Selecting and Retrieving Data

7824888

- pandas
  - pandas Data Structures
  - Exploring DataFrames
  - Selecting and Retrieving Data in DataFrames
  - Filtering Data
  - DataFrame Modification
  - Aggregation in DataFrame
  - Joining DataFrames
- Visualization
  - Matplotlib
  - Plotly

Integer-based Extraction: iloc[]

7824888

We want to extract data according to its *position*.

Row position		0	1	2	3
		Landmark	Type	Height	Year Built
0	0	Sather Gate	Gate	30	1910.0
1	1	Campanile	Tower	307	1914.0
2	2	Doe Library	Library	80	1911.0
3	3	Memorial Glade	Open Space	0	NaN
4	4	Sproul Plaza	Plaza	0	1962.0

Column position

Python convention: The first position has integer index 0.

Arguments to .`iloc` can be:

- A list.
- A slice (syntax is **exclusive** of the right-hand side of the slice).
- A single value.

Integer-based Extraction: iloc[]

7824888

Arguments to .iloc can be:

- A list.
- A slice (syntax is **exclusive** of the right-hand side of the slice).
- A single value.

		0	1	2	3
		Landmark	Type	Height	Year Built
0	0	Sather Gate	Gate	30	1910.0
1	1	Campanile	Tower	307	1914.0
2	2	Doe Library	Library	80	1911.0
3	3	Memorial Glade	Open Space	0	NaN
4	4	Sproul Plaza	Plaza	0	1962.0

Integer-based Extraction: iloc[]

7824888

Arguments to .iloc can be:

- **A list.**
- A slice (syntax is **exclusive** of the right-hand side of the slice).
- A single value.

```
df.iloc[[1, 2]]
```

	Landmark	Type	Height	Year Built
1	Campanile	Tower	307	1914.0
2	Doe Library	Library	80	1911.0

```
df.iloc[[1, 2], [1, 2, 3]]
```

	Type	Height	Year Built
1	Tower	307	1914.0
2	Library	80	1911.0

		0	1	2	3
		Landmark	Type	Height	Year Built
0	0	Sather Gate	Gate	30	1910.0
1	1	Campanile	Tower	307	1914.0
2	2	Doe Library	Library	80	1911.0
3	3	Memorial Glade	Open Space	0	NaN
4	4	Sproul Plaza	Plaza	0	1962.0

Integer-based Extraction: iloc[]

7824888

Arguments to .iloc can be:

- A list.
- **A slice** (syntax is **exclusive** of the right-hand side of the slice).
- A single value.

```
df.iloc[2:4]
```

	Landmark	Type	Height	Year Built
2	Doe Library	Library	80	1911.0
3	Memorial Glade	Open Space	0	NaN

```
df.iloc[:, 1:2]
```

```
0          Gate
1         Tower
2       Library
3    Open Space
4         Plaza
Name: Type, dtype: object
```

	0	1	2	3
	Landmark	Type	Height	Year Built
0	0 Sather Gate	Gate	30	1910.0
1	1 Campanile	Tower	307	1914.0
2	2 Doe Library	Library	80	1911.0
3	3 Memorial Glade	Open Space	0	NaN
4	4 Sproul Plaza	Plaza	0	1962.0

Integer-based Extraction: iloc[]

7824888

Arguments to .iloc can be:

- A list.
- A slice (syntax is **exclusive** of the right-hand side of the slice).
- **A single value**.

```
df.iloc[:, 0]
```

```
0       Sather Gate
1         Campanile
2       Doe Library
3    Memorial Glade
4       Sproul Plaza
Name: Landmark, dtype: object
```

	0	1	2	3
	Landmark	Type	Height	Year Built
0	Sather Gate	Gate	30	1910.0
1	Campanile	Tower	307	1914.0
2	Doe Library	Library	80	1911.0
3	Memorial Glade	Open Space	0	NaN
4	Sproul Plaza	Plaza	0	1962.0

```
df.iloc[2]
```

```
Landmark      Doe Library
Type              Library
Height                 80
Year Built         1911.0
Name: 2, dtype: object
```

```
df.iloc[0, 1]
```

```
Landmark      Doe Library
Type              Library
Height                 80
Year Built         1911.0
Name: 2, dtype: object
```

Label-based Extraction: loc[]

7824888

We want to extract data according to its *labels*.

Row Labels

	Landmark	Type	Height	Year Built
0	Sather Gate	Gate	30	1910.0
1	Campanile	Tower	307	1914.0
2	Doe Library	Library	80	1911.0
3	Memorial Glade	Open Space	0	NaN
4	Sproul Plaza	Plaza	0	1962.0

Column Labels

Arguments to `.loc` can be:

- A list.
- A slice (syntax is **inclusive** of the right-hand side of the slice).
- A single value.

Label-based Extraction: loc[]

7824888

Arguments to .loc can be:

- A list.
- A slice (syntax is **inclusive** of the right-hand side of the slice).
- A single value.

Column Labels

Row Labels

	Landmark	Type	Height	Year Built
0	Sather Gate	Gate	30	1910.0
1	Campanile	Tower	307	1914.0
2	Doe Library	Library	80	1911.0
3	Memorial Glade	Open Space	0	NaN
4	Sproul Plaza	Plaza	0	1962.0

Label-based Extraction: loc[]

7824888

Arguments to .loc can be:

- **A list.**
- A slice (syntax is **inclusive** of the right-hand side of the slice).
- A single value.

```
df.loc[[1, 2]]
```

	Landmark	Type	Height	Year Built
1	Campanile	Tower	307	1914.0
2	Doe Library	Library	80	1911.0

```
df.loc[[1, 2], ['Type', 'Height', 'Year Built']]
```

	Type	Height	Year Built
1	Tower	307	1914.0
2	Library	80	1911.0

Row Labels

Column Labels

	Landmark	Type	Height	Year Built
0	Sather Gate	Gate	30	1910.0
1	Campanile	Tower	307	1914.0
2	Doe Library	Library	80	1911.0
3	Memorial Glade	Open Space	0	NaN
4	Sproul Plaza	Plaza	0	1962.0

Label-based Extraction: loc[]

7824888

Arguments to .loc can be:

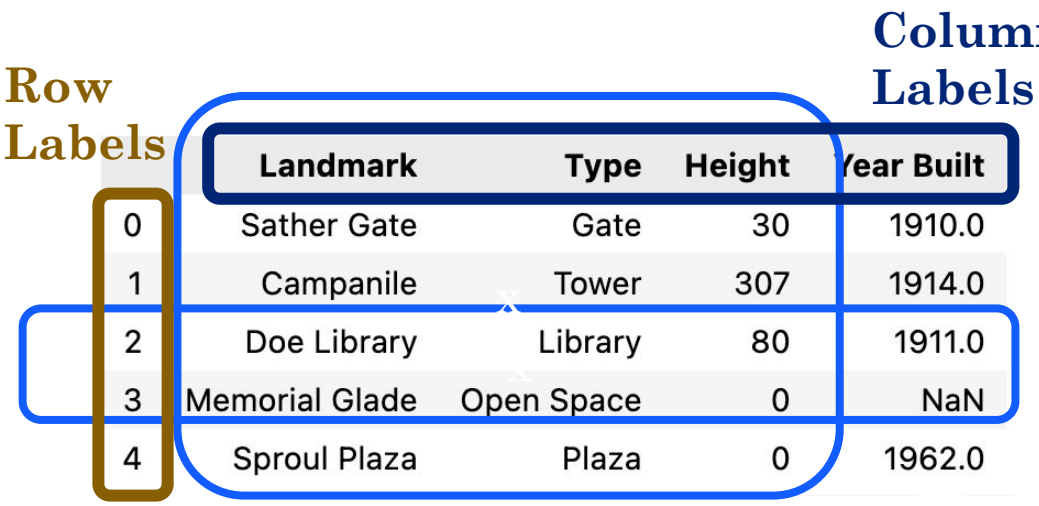
- A list.
- **A slice** (syntax is **inclusive** of the right-hand side of the slice).
- A single value.

```
df.loc[2:3]
```

	Landmark	Type	Height	Year Built
2	Doe Library	Library	80	1911.0
3	Memorial Glade	Open Space	0	NaN

```
df.loc[:, 'Landmark':'Height']
```

	Landmark	Type	Height
0	Sather Gate	Gate	30
1	Campanile	Tower	307
2	Doe Library	Library	80
3	Memorial Glade	Open Space	0
4	Sproul Plaza	Plaza	0



Label-based Extraction: loc[]

7824888

Arguments to .loc can be:

- A list.
- A slice (syntax is **inclusive** of the right-hand side of the slice).
- **A single value**.

```
df.loc[:, 'Landmark']
```

```
0       Sather Gate
1         Campanile
2       Doe Library
3    Memorial Glade
4       Sproul Plaza
Name: Landmark, dtype: object
```

```
df.loc[2]
```

```
Landmark      Doe Library
Type              Library
Height                 80
Year Built         1911.0
Name: 2, dtype: object
```

```
df.loc[0, 'Type']
```

```
Gate
```

Row Labels

Column Labels

	Landmark	Type	Height	Year Built
0	Sather Gate	Gate	30	1910.0
1	Campanile	Tower	307	1914.0
2	Doe Library	Library	80	1911.0
3	Memorial Glade	Open Space	0	NaN
4	Sproul Plaza	Plaza	0	1962.0

Context-Dependent Selection

7824888

- [ ] only takes one argument, which may be:
  - A slice of **row numbers**.
  - A list of **column labels**.
  - A single **column label**.

That is, [ ] is context sensitive.

Context-Dependent Selection

7824888

- [ ] only takes one argument, which may be:
  - **A slice of row numbers.**
  - A list of column labels.
  - A single column label.

```
df[1:3]
```

	Landmark	Type	Height	Year Built
1	Campanile	Tower	307	1914.0
2	Doe Library	Library	80	1911.0

	Landmark	Type	Height	Year Built
0	Sather Gate	Gate	30	1910.0
1	Campanile	Tower	307	1914.0
2	Doe Library	Library	80	1911.0
3	Memorial Glade	Open Space	0	NaN
4	Sproul Plaza	Plaza	0	1962.0

Context-Dependent Selection

7824888

- [ ] only takes one argument, which may be:
  - A slice of row numbers.
  - **A list of column labels.**
  - A single column label.

```
df[['Landmark', 'Year Built']]
```

	Landmark	Year Built
0	Sather Gate	1910.0
1	Campanile	1914.0
2	Doe Library	1911.0
3	Memorial Glade	NaN
4	Sproul Plaza	1962.0

	Landmark	Type	Height	Year Built
0	Sather Gate	Gate	30	1910.0
1	Campanile	Tower	307	1914.0
2	Doe Library	Library	80	1911.0
3	Memorial Glade	Open Space	0	NaN
4	Sproul Plaza	Plaza	0	1962.0

Context-Dependent Selection

7824888

- [ ] only takes one argument, which may be:
  - A slice of row numbers.
  - A list of column labels.
  - **A single column label.**

```
df['Year Built']
```

```
0    1910.0
1    1914.0
2    1911.0
3       NaN
4    1962.0
Name: Year Built, dtype: float64
```

	Landmark	Type	Height	Year Built
0	Sather Gate	Gate	30	1910.0
1	Campanile	Tower	307	1914.0
2	Doe Library	Library	80	1911.0
3	Memorial Glade	Open Space	0	NaN
4	Sproul Plaza	Plaza	0	1962.0

Which landmark name is returned by each command?

1. df.loc[1, "Landmark"]

2. df.iloc[2, 0]

3. df["Landmark"][3]

Filtering Data

7824888

- pandas
  - pandas Data Structures
  - Exploring DataFrames
  - Selecting and Retrieving Data in DataFrames
  - Filtering Data
  - DataFrame Modification
  - Aggregation in DataFrame
  - Joining DataFrames
- Visualization
  - Matplotlib
  - Plotly

Boolean Array for Filtering a `DataFrame`

7824888

We learned to extract data according to its **integer position** (`.iloc`) or its **label** (`.loc`)

What if we want to extract rows that satisfy a given *condition*?

- `.loc` and `[ ]` also accept Boolean arrays as input.
- Rows corresponding to `True` are extracted; rows corresponding to `False` are not.

```
df[df['Height'] > 50]
```

	Landmark	Type	Height	Year Built
0	Sather Gate	Gate	30	1910.0
1	Campanile	Tower	307	1914.0
2	Doe Library	Library	80	1911.0
3	Memorial Glade	Open Space	0	NaN
4	Sproul Plaza	Plaza	0	1962.0

```
0    False
1     True
2     True
3    False
4    False
Name: Height, dtype: bool
```

	Landmark	Type	Height	Year Built
1	Campanile	Tower	307	1914.0
2	Doe Library	Library	80	1911.0

```
df['Height'] > 50
```

equivalently `df.loc[df['Height'] > 50]`

Combining Boolean Series for Filtering

7824888

Boolean `Series` can be combined using various operators, allowing filtering of results by multiple criteria.

- The `&` operator allows us to apply `operand_1` ***and*** `operand_2`
- The `|` operator allows us to apply `operand_1` ***or*** `operand_2`

```
df[(df['Height'] > 50) & (df['Type'] == 'Library')]
```

	Landmark	Type	Height	Year Built
0	Sather Gate	Gate	30	1910.0
1	Campanile	Tower	307	1914.0
2	Doe Library	Library	80	1911.0
3	Memorial Glade	Open Space	0	NaN
4	Sproul Plaza	Plaza	0	1962.0

	Landmark	Type	Height	Year Built
2	Doe Library	Library	80	1911.0

Bitwise Operators

7824888

- `&` and `|` are examples of **bitwise operators**. They allow us to apply multiple logical conditions.
- If `p` and `q` are boolean arrays or `Series`:

Symbol	Usage	Meaning
`~`	`~p`	Negation of `p`
`|`	`p | q`	`p` OR `q`
`&`	`p & q`	`p` AND `q`
`^`	`p ^ q`	`p` XOR `q` (exclusive or)

DataFrame Modification

7824888

- pandas
  - pandas Data Structures
  - Exploring DataFrames
  - Selecting and Retrieving Data in DataFrames
  - Filtering Data
  - DataFrame Modification
  - Aggregation in DataFrame
  - Joining DataFrames
- Visualization
  - Matplotlib
  - Plotly

Adding and Modifying Columns

7824888

Adding a column is easy:

1. Use [ ] to reference the desired new column.
2. Assign this column to a `Series` or array of the appropriate length.

```
df['Experience'] = [2, 5, 1, 8, 4]
```

	Landmark	Type	Height	Year Built
0	Sather Gate	Gate	30	1910.0
1	Campanile	Tower	307	1914.0
2	Doe Library	Library	80	1911.0
3	Memorial Glade	Open Space	0	NaN
4	Sproul Plaza	Plaza	0	1962.0

	Landmark	Type	Height	Year Built	Experience
0	Sather Gate	Gate	30	1910.0	2
1	Campanile	Tower	307	1914.0	5
2	Doe Library	Library	80	1911.0	1
3	Memorial Glade	Open Space	0	NaN	8
4	Sproul Plaza	Plaza	0	1962.0	4

```
df['Height_Increase'] = df['Height'] * 0.1
```

	Landmark	Type	Height	Year Built	Experience	Height_Increase
0	Sather Gate	Gate	30	1910.0	2	3.0
1	Campanile	Tower	307	1914.0	5	30.7
2	Doe Library	Library	80	1911.0	1	8.0
3	Memorial Glade	Open Space	0	NaN	8	0.0
4	Sproul Plaza	Plaza	0	1962.0	4	0.0

Dropping a Column

7824888

```
df.drop(columns=['Experience'])

print(df)
```

	Landmark	Type	Height	Year Built	Experience	Height_Increase
0	Sather Gate	Gate	30	1910.0	2	3.0
1	Campanile	Tower	307	1914.0	5	30.7
2	Doe Library	Library	80	1911.0	1	8.0
3	Memorial Glade	Open Space	0	NaN	8	0.0
4	Sproul Plaza	Plaza	0	1962.0	4	0.0

	Landmark	Type	Height	Year Built	Experience	Height_Increase
0	Sather Gate	Gate	30	1910.0	2	3.0
1	Campanile	Tower	307	1914.0	5	30.7
2	Doe Library	Library	80	1911.0	1	8.0
3	Memorial Glade	Open Space	0	NaN	8	0.0
4	Sproul Plaza	Plaza	0	1962.0	4	0.0

What happened?

Most `DataFrame` operations are not in-place.

Dropping a Column

7824888

```
df.drop(columns=['Experience'], inplace=True)

print(df)
```

	Landmark	Type	Height	Year Built	Experience	Height_Increase
0	Sather Gate	Gate	30	1910.0	2	3.0
1	Campanile	Tower	307	1914.0	5	30.7
2	Doe Library	Library	80	1911.0	1	8.0
3	Memorial Glade	Open Space	0	NaN	8	0.0
4	Sproul Plaza	Plaza	0	1962.0	4	0.0

	Landmark	Type	Height	Year Built	Height_Increase
0	Sather Gate	Gate	30	1910.0	5.0
1	Campanile	Tower	307	1914.0	30.7
2	Doe Library	Library	80	1911.0	8.0
3	Memorial Glade	Open Space	0	NaN	0.0
4	Sproul Plaza	Plaza	0	1962.0	0.0

Alternatively we can directly assign:

```
df = df.drop(columns=['Experience'])
```

Sorting `DataFrame`

7824888

- Sorting organizes your data for better analysis.
- We use `sort_values()` to sort by one or more columns in ascending or descending order.
- Syntax:
  - **Single column**:

```
df.sort_values(by='Column', ascending=True)
```

  - **Multiple columns**:

```
df.sort_values(by=['Col1', 'Col2'], ascending=[True, False])
```

Sorting `DataFrame` by One Column

7824888

- **Single column**:

```
df.sort_values(by='Column', ascending=True)
```

```
df = df.sort_values(by='Height')
```

The default value for the ascending argument is `True`.

	Landmark	Type	Height	Year Built
0	Sather Gate	Gate	30	1910.0
1	Campanile	Tower	307	1914.0
2	Doe Library	Library	80	1911.0
3	Memorial Glade	Open Space	0	NaN
4	Sproul Plaza	Plaza	0	1962.0

	Landmark	Type	Height	Year Built
3	Memorial Glade	Open Space	0	NaN
4	Sproul Plaza	Plaza	0	1962.0
0	Sather Gate	Gate	30	1910.0
2	Doe Library	Library	80	1911.0
1	Campanile	Tower	307	1914.0

Sorting `DataFrame` by One Column

7824888

- **Multiple columns**:

```
df.sort_values(by=['Col1', 'Col2'],
               ascending=[True, True])
```

```
df = df.sort_values(by=['Height', 'Type'], ascending=[True, False]))
```

	Landmark	Type	Height	Year Built
0	Sather Gate	Gate	30	1910.0
1	Campanile	Tower	307	1914.0
2	Doe Library	Library	80	1911.0
3	Memorial Glade	Open Space	0	NaN
4	Sproul Plaza	Plaza	0	1962.0

	Landmark	Type	Height	Year Built
4	Sproul Plaza	Plaza	0	1962.0
3	Memorial Glade	Open Space	0	NaN
0	Sather Gate	Gate	30	1910.0
2	Doe Library	Library	80	1911.0
1	Campanile	Tower	307	1914.0

If you run df.sort_values('Height', ascending=False).iloc[0]['Landmark'], which landmark do you get?

Handling Missing Values

7824888

- Missing values are a common issue in real-world datasets.
- We will explore techniques to:
  - Detect missing values.
  - Handle missing values by either **removing** or **imputing** them.

Handling Missing Values

7824888

- We will explore techniques to:
  - Detect missing values.

```
df_missing.isnull()
```

	Landmark	Type	Height	Year Built
0	Sather Gate	Gate	30.0	NaN
1	Campanile	Tower	307.0	1914.0
2	Doe Library	Library	NaN	1911.0
3	Memorial Glade	Open Space	0.0	NaN
4	Sproul Plaza	None	0.0	1962.0

	Landmark	Type	Height	Year Built
0	False	False	False	True
1	False	False	False	False
2	False	False	True	False
3	False	False	False	True
4	False	True	False	False

Handling Missing Values

7824888

- We will explore techniques to:
  - Handle missing values by either **removing** or imputing them.

```
df_missing.dropna()
```

	Landmark	Type	Height	Year Built
0	Sather Gate	Gate	30.0	NaN
1	Campanile	Tower	307.0	1914.0
2	Doe Library	Library	NaN	1911.0
3	Memorial Glade	Open Space	0.0	NaN
4	Sproul Plaza	None	0.0	1962.0

	Landmark	Type	Height	Year Built
1	Campanile	Tower	307.0	1914.0

Handling Missing Values

7824888

- We will explore techniques to:
  - Handle missing values by either removing or **imputing** them.

```
df_missing.fillna()
```

	Landmark	Type	Height	Year Built
0	Sather Gate	Gate	30.0	NaN
1	Campanile	Tower	307.0	1914.0
2	Doe Library	Library	NaN	1911.0
3	Memorial Glade	Open Space	0.0	NaN
4	Sproul Plaza	None	0.0	1962.0

	Landmark	Type	Height	Year Built
0	Sather Gate	Gate	30.0	0.0
1	Campanile	Tower	307.0	1914.0
2	Doe Library	Library	0.0	1911.0
3	Memorial Glade	Open Space	0.0	0.0
4	Sproul Plaza	0	0.0	1962.0

Aggregation in DataFrame

7824888

- pandas
  - pandas Data Structures
  - Exploring DataFrames
  - Selecting and Retrieving Data in DataFrames
  - Filtering Data
  - DataFrame Modification
  - Aggregation in DataFrame
  - Joining DataFrames
- Visualization
  - Matplotlib
  - Plotly

Aggregating Data in pandas

7824888

- We can perform aggregations on our data, such as calculating means, sums, and other summary statistics.

Basic

`sum()`
`mean()`
`median()`
`min()`
`max()`
`count()`
`nunique()`– # of unique values
`prod()`

```
df['Height'].mean()
```

```
83.4
```

Statistical

`std()`
`var()`
`sem()`– standard error of the mean
`skew()`

```
df['Height'].std()
```

```
129.2006191935627
```

Logical and Index-Based

`any()`– `True` if **any** value is `True`
`all()`– `True` if **all** values are `True`

`first()`– first non-null value
`last()`– last non-null value
`idxmin()`– index of min value
`idxmax()`– index of max value

```
df['Height'].idxmax()
```

```
1
```

Grouping Data in pandas

7824888

Our goal:

- Group together rows that fall under the **same category**.
  - For example, group together all rows representing a Tower landmark.
- Perform an operation that ***aggregates*** across all rows in the category.
  - For example, sum up the average height of all Tower landmarks.

Grouping is a powerful tool to 1) perform large operations, all at once and 2) summarize trends in a dataset.

Grouping Data in pandas

7824888

A `.groupby()` operation involves some combination of **splitting the object, applying a function**, and **combining the results**.

	Landmark	Type	Height	Year Built	Campus
0	Sather Gate	Gate	30	1910.0	UC Berkeley
1	Campanile	Tower	307	1914.0	UC Berkeley
2	Doe Library	Library	80	1911.0	UC Berkeley
3	Memorial Glade	Open Space	0	NaN	UC Berkeley
4	Sproul Plaza	Plaza	0	1962.0	UC Berkeley
5	North Gate	Gate	25	1909.0	UC Berkeley
6	Moffitt Library	Library	60	1970.0	UC Berkeley
7	Faculty Glade	Open Space	0	NaN	UC Berkeley
8	Lower Sproul Plaza	Plaza	0	2015.0	UC Berkeley
9	77 Mass Ave Entrance	Gate	15	1939.0	MIT
10	Green Building	Tower	295	1964.0	MIT
11	Barker Library	Library	0	1916.0	MIT
12	Killian Court	Open Space	0	1920.0	MIT
13	Stata Center Courtyard	Plaza	0	2004.0	MIT
14	Palm Drive Entrance	Gate	20	1890.0	Stanford
15	Hoover Tower	Tower	285	1941.0	Stanford
16	Green Library	Library	0	1919.0	Stanford
17	Main Quad	Open Space	0	1891.0	Stanford
18	White Plaza	Plaza	0	1964.0	Stanford

Grouping Data in pandas

7824888

```
augmented_df.groupby('Type')
```

	Landmark	Type	Height	Year Built	Campus
0	Sather Gate	Gate	30	1910.0	UC Berkeley
1	Campanile	Tower	307	1914.0	UC Berkeley
2	Doe Library	Library	80	1911.0	UC Berkeley
3	Memorial Glade	Open Space	0	NaN	UC Berkeley
4	Sproul Plaza	Plaza	0	1962.0	UC Berkeley
5	North Gate	Gate	25	1909.0	UC Berkeley
6	Moffitt Library	Library	60	1970.0	UC Berkeley
7	Faculty Glade	Open Space	0	NaN	UC Berkeley
8	Lower Sproul Plaza	Plaza	0	2015.0	UC Berkeley
9	77 Mass Ave Entrance	Gate	15	1939.0	MIT
10	Green Building	Tower	295	1964.0	MIT
11	Barker Library	Library	0	1916.0	MIT
12	Killian Court	Open Space	0	1920.0	MIT
13	Stata Center Courtyard	Plaza	0	2004.0	MIT
14	Palm Drive Entrance	Gate	20	1890.0	Stanford
15	Hoover Tower	Tower	285	1941.0	Stanford
16	Green Library	Library	0	1919.0	Stanford
17	Main Quad	Open Space	0	1891.0	Stanford
18	White Plaza	Plaza	0	1964.0	Stanford

DataFrameGroupBy

Gate

	Landmark	Type	Height	Year Built	Campus
0	Sather Gate	Gate	30	1910.0	UC Berkeley
5	North Gate	Gate	25	1909.0	UC Berkeley
9	77 Mass Ave Entrance	Gate	15	1939.0	MIT
14	Palm Drive Entrance	Gate	20	1890.0	Stanford

Tower

	Landmark	Type	Height	Year Built	Campus
1	Campanile	Tower	307	1914.0	UC Berkeley
10	Green Building	Tower	295	1964.0	MIT
15	Hoover Tower	Tower	285	1941.0	Stanford

Plaza

	Landmark	Type	Height	Year Built	Campus
4	Sproul Plaza	Plaza	0	1962.0	UC Berkeley
8	Lower Sproul Plaza	Plaza	0	2015.0	UC Berkeley
13	Stata Center Courtyard	Plaza	0	2004.0	MIT
18	White Plaza	Plaza	0	1964.0	Stanford

Open Space

	Landmark	Type	Height	Year Built	Campus
3	Memorial Glade	Open Space	0	NaN	UC Berkeley
7	Faculty Glade	Open Space	0	NaN	UC Berkeley
12	Killian Court	Open Space	0	1920.0	MIT
17	Main Quad	Open Space	0	1891.0	Stanford

Library

	Landmark	Type	Height	Year Built	Campus
2	Doe Library	Library	80	1911.0	UC Berkeley
6	Moffitt Library	Library	60	1970.0	UC Berkeley
11	Barker Library	Library	0	1916.0	MIT
16	Green Library	Library	0	1919.0	Stanford

```
.agg(mean)
```

Type	Height	Year Built
Gate	22.500000	1912.000000
Tower	295.666667	1939.666667
Plaza	0.000000	1986.250000
Open Space	0.000000	1905.500000
Library	35.000000	1929.000000

Aggregation Functions

7824888

What goes inside of `.agg( )`?Any function that aggregates several values into one summary value.

- Common examples:

In-Built Python Functions

```
.agg(sum)
.agg(max)
.agg(min)
```

`NumPy` Functions

```
.agg(np.sum)
.agg(np.max)
.agg(np.min)
.agg(np.mean)
```

In-Built `pandas` Functions

```
.agg("sum")
.agg("max")
.agg("min")
.agg("mean")
.agg("first")
.agg("last")
```

Some commonly-used aggregation functions can even be called directly, without the explicit use of `.agg( )`:

```
df.groupby("Type").mean()
```

What will df.groupby('Type')['Height'].mean()['Plaza'] return?

Grouping by Multiple Columns

7824888

```
augmented_df.groupby(['Type', 'Campus'])[['Height']].agg('max')
```

Grouping by Multiple Columns: `pivot_table`

7824888

```
augmented_df.groupby(['Type', 'Campus'])[['Height']].agg('max')
```

Type	Campus	Height
Gate	MIT	15
	Stanford	20
	UC Berkeley	30
Tower	MIT	295
	Stanford	285
	UC Berkeley	307
Plaza	MIT	0
	Stanford	0
	UC Berkeley	0
Open Space	MIT	0
	Stanford	0
	UC Berkeley	0
Library	MIT	0
	Stanford	0
	UC Berkeley	80

rows columns values

But we have two index in our `DataFrame`

```
pivot_table =
    pd.pivot_table(
        augmented_df,
        index='Type',
        columns='Campus',
        values='Height',
        aggfunc='max'
    )
```

Joining DataFrames

7824888

- pandas
  - pandas Data Structures
  - Exploring DataFrames
  - Selecting and Retrieving Data in DataFrames
  - Filtering Data
  - DataFrame Modification
  - Aggregation in DataFrame
  - Joining DataFrames
- Visualization
  - Matplotlib
  - Plotly

7824888

Joining Events and Landmarks

Joining `DataFrames` allows you to combine data from different sources based on a common key or index.

`event_data`

	Year	Event	Location
0	1868	Founding of UC Berkeley	Berkeley, CA
1	1914	Completion of Campanile	Berkeley, CA
2	1923	Opening of Memorial Stadium	Berkeley, CA
3	1964	Free Speech Movement	Berkeley, CA
4	2000	Opening of Hearst Memorial Mining Building	Berkeley, CA

`landmarks`

	Landmark	Type	Height	Year Built
0	Sather Gate	Gate	30	1910.0
1	Campanile	Tower	307	1914.0
2	Doe Library	Library	80	1911.0
3	Memorial Glade	Open Space	0	NaN
4	Sproul Plaza	Plaza	0	1962.0





Types of Join: Inner Join

7824888

- **Inner Join**: Returns only the rows with matching keys in both `DataFrames`.

`landmarks`

	Landmark	Type	Height	Year Built
0	Sather Gate	Gate	30	1910.0
1	Campanile	Tower	307	1914.0
2	Doe Library	Library	80	1911.0
3	Memorial Glade	Open Space	0	NaN
4	Sproul Plaza	Plaza	0	1962.0

`event_data`

	Year	Event	Location
0	1868	Founding of UC Berkeley	Berkeley, CA
1	1914	Completion of Campanile	Berkeley, CA
2	1923	Opening of Memorial Stadium	Berkeley, CA
3	1964	Free Speech Movement	Berkeley, CA
4	2000	Opening of Hearst Memorial Mining Building	Berkeley, CA

```
result_join_inner = landmarks.join(event_data.set_index('Year'),
            on= 'Year Built',
            how='inner')

result_merge_inner = landmarks.merge(event_data,
                    how='inner',
                    left_on='Year Built',
                    right_on='Year')
```

	Landmark	Type	Height	Year Built	Year	Event	Location
0	Campanile	Tower	307	1914.0	1914	Completion of Campanile	Berkeley, CA

Types of Join: Outer Join

7824888

- **Outer Join**: Returns all rows from both `DataFrames`, filling missing values with `NaN` where there is no match.

`landmarks`

	Landmark	Type	Height	Year Built
0	Sather Gate	Gate	30	1910.0
1	Campanile	Tower	307	1914.0
2	Doe Library	Library	80	1911.0
3	Memorial Glade	Open Space	0	NaN
4	Sproul Plaza	Plaza	0	1962.0

no match

no match

`event_data`

	Year	Event	Location
0	1868	Founding of UC Berkeley	Berkeley, CA
1	1914	Completion of Campanile	Berkeley, CA
2	1923	Opening of Memorial Stadium	Berkeley, CA
3	1964	Free Speech Movement	Berkeley, CA
4	2000	Opening of Hearst Memorial Mining Building	Berkeley, CA

```
result_merge_outer = landmarks.merge(event_data,
                    how='outer',
                    left_on='Year Built',
                    right_on='Year')
```

Types of Join

7824888

- **Inner Join**: Returns only the rows with matching keys in both `DataFrames`.
- **Outer Join**: Returns all rows from both `DataFrames`, filling missing values with `NaN` where there is no match.
- **Left Join**: Returns all rows from the left `DataFrame` and matching rows from the right `DataFrame`.
- **Right Join**: Returns all rows from the right `DataFrame` and matching rows from the left `DataFrame`.

Visualization

7824888

- pandas
  - pandas Data Structures
  - Exploring DataFrames
  - Selecting and Retrieving Data in DataFrames
  - Filtering Data
  - DataFrame Modification
  - Aggregation in DataFrame
  - Joining DataFrames
- Visualization
  - Matplotlib
  - Plotly

Why Do We Visualize Data?

7824888

Insight: Critical to gaining deeper insights into trends in the data.

Communication: Help convey trends in data to others.

In this class we will explore both but focus more on the first.

Plotting Libraries

7824888

There are a wide range of tools for data visualization in machine learning.

- **Weights and Biases** – Commercial service used for tracking training runs and model artifacts.
- **`Matplotlib`** – Python library commonly used for static plots.
- **`Plotly`** – Cross-language interactive plotting library.

In this course we will focus on **`Plotly`** and **Weights and Biases**.

Matplotlib

7824888

- pandas
  - pandas Data Structures
  - Exploring DataFrames
  - Selecting and Retrieving Data in DataFrames
  - Filtering Data
  - DataFrame Modification
  - Aggregation in DataFrame
  - Joining DataFrames
- Visualization
  - Matplotlib
  - Plotly

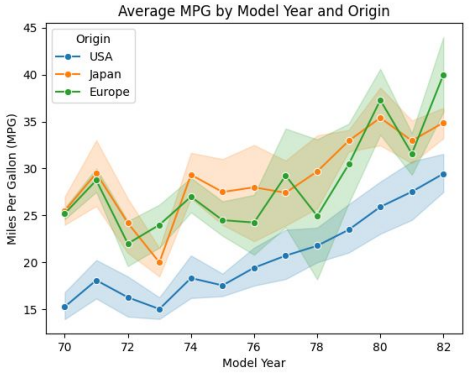
Matplotlib

7824888

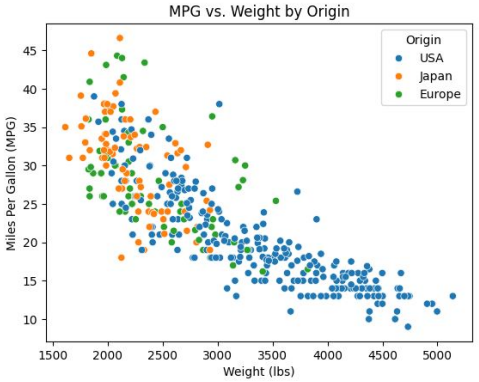
- `Matplotlib` is a versatile Python library for creating static and publication-quality visualizations.

```
import matplotlib.pyplot as plt
```

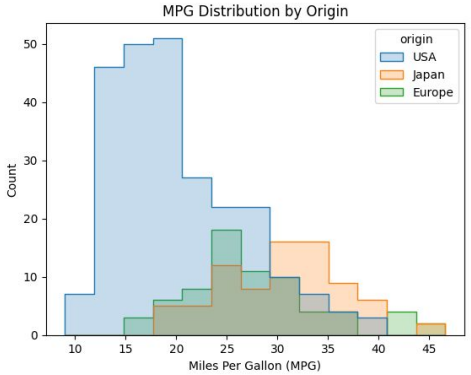
Line Plot



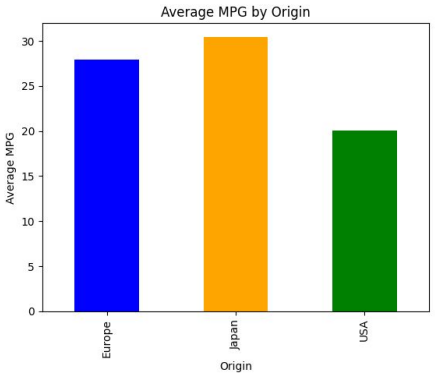
Scatter Plot



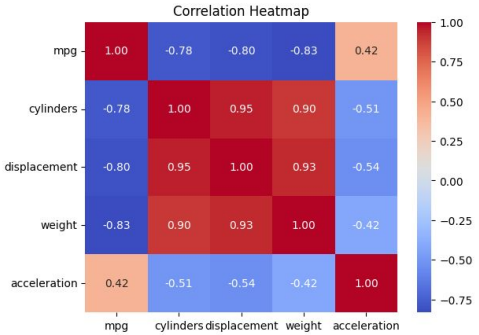
Histogram



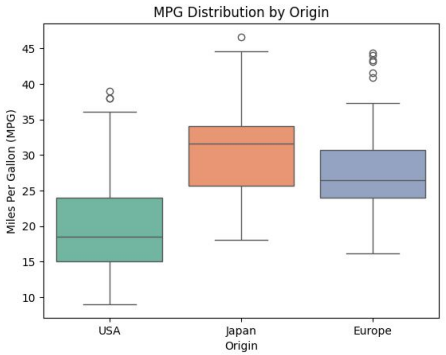
Bar Plot



Heatmap



Box Plot



Plotly

- pandas
  - pandas Data Structures
  - Exploring DataFrames
  - Selecting and Retrieving Data in DataFrames
  - Filtering Data
  - DataFrame Modification
  - Aggregation in DataFrame
  - Joining DataFrames
- Visualization
  - Matplotlib
  - Plotly

Why `Plotly`?

7824888

Many would (*correctly*) argue **you should learn `MatplotLib`**.

- **Gold standard** for python visualization since ~2005.
- Most paper plots are still made with `Matplotlib`.
- If we had time we would teach both (we may use some in demos).

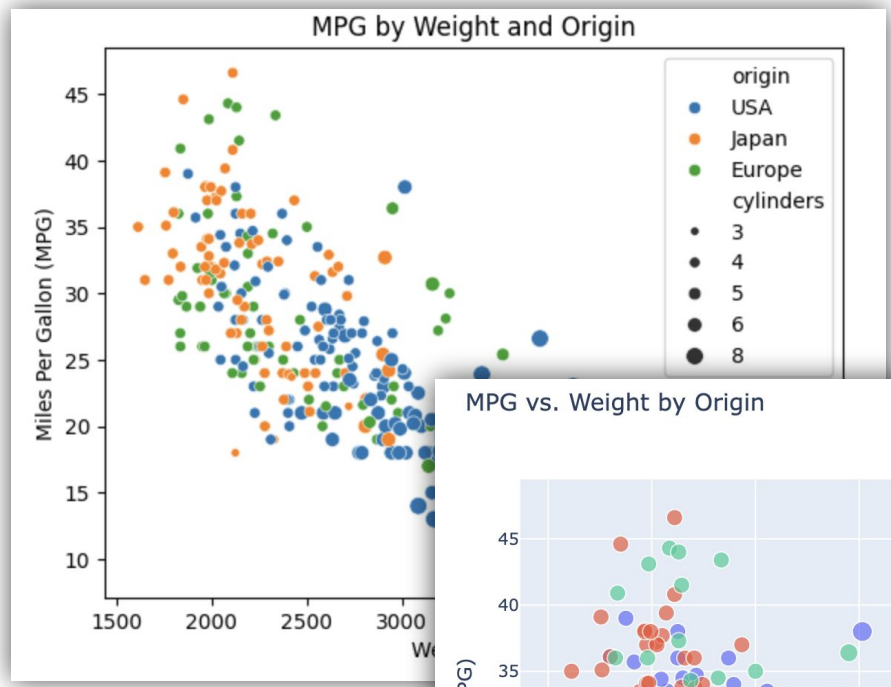
However, **`Matplotlib` plots are static** making it difficult to interactively slice.

- Weights and Bias and `Plotly` are **designed for interaction.**
- **Quicker to gain insights** – a focus of this class.

Demo

Matplotlib vs plotly

7824888



Three Ways to Plot using `Plotly`

7824888

Easiest: Using **`pandas` built-in** plotting functions

- Great place to start.

Easy + Expressive: Use **`Plotly Express`** to construct plots quickly

- Like pandas plotting functions but with more options.

Advanced: Build plot from graphics objects (like `Matplotlib`)

- Need to learn some basic `Plotly` concepts.

Using pandas built-in Plotting Functions

7824888

Configure pandas to use Plotly (done at beginning of notebook).

```
pd.set_option('plotting.backend', 'plotly')
```

Call plot directly on your DataFrame:

```
# Make various kinds of plots 'scatter', 'bar', 'hist'
mpg.plot(kind='scatter', x='weight', y='mpg', color='origin',
        title='MPG vs. Weight by Origin',
        width=800, height=600)
```

Notice that we define:

- The **kind** of plot (e.g., 'scatter', 'bar')
- How columns are attached to visual elements (e.g., x, y, color, size, shape …)

Using `Plotly Express`

7824888

Very similar to calling `pandas .plot` functions but with a wide range of plotting capabilities (see tutorials):

```
import plotly.express as px
px.scatter(mpg, x='weight', y='mpg', color='origin', size='cylinders',
     hover_data = mpg.columns,
     title='MPG vs. Weight by Origin',
     width=800, height=600)
```

Here we pass the `DataFrame` (`mpg`) into the desired plotting function along with how columns are mapped to visual elements.

With:

px.scatter(df, x="Year Built", y="Height", size="Height", hover_name="Landmark")

What happens to the marker for Campanile (Height=307, Year Built=1914)?

How you Can Learn More

7824888

Skim the tutorials (optional but can be fun):

- Plotly Express Overview
- Scatter Plots
- Line Plots
- Bar Plots
- Histograms and Dist Plots
- Facet Plots
- Subplots
- Discrete and Continuous Color
- Formatting Axes
- Graphics Objects

When you can't figure out how to plot something (or just want to learn more) ask an AI agent.

- Most LLMs are very good at `Plotly` and `Matplotlib`.

Lecture 2

Data Tools

Credit: Joseph E. Gonzalez and Narges Norouzi

Reference Book Chapters: This topic is not covered in the textbook.